Safety-Preserving Compression Between Generator and Safety Critic: Measuring and Minimizing Safety Loss in Prompt-based PRM

Aidan Pinero
UC Santa Cruz
apinero@ucsc.edu

Glenn Grant-Richards
UC Santa Cruz
ggrantri@ucsc.edu

Sophia Yang
UC Santa Cruz
syang255@ucsc.edu

Abstract

Large language model agents generate verbose reasoning traces that can strain computational resources and exceed context window limits of downstream safety critics. We investigate whether compressing these traces before safety evaluation introduces "safety loss"—the degradation in a critic's ability to detect unsafe behaviors. Our pipeline uses entropy-based token pruning (GPT-2) to compress agent outputs before evaluation by Llama Guard 3. Preliminary experiments on AgentDojo (N=10) reveal a surprising finding: compression at ratios $\geq$50% does not cause the critic to miss attacks (0% attack success rate), but instead triggers severe over-refusal, with 90% of content incorrectly flagged as unsafe. This suggests fragmented text from aggressive token pruning produces false safety alarms rather than enabling attacks to slip through undetected, highlighting a critical usability-safety tradeoff for compressed agent systems.

1 Approach

1.1 Generator-Compressor-Critic Pipeline

We implement a three-stage pipeline to measure safety loss under compression. The **Generator** produces structured agent traces containing system prompts, user queries, and conversation history from the AgentDojo dataset (**?**). These traces are fed to an **Entropy-Based Compressor**, which calculates per-token perplexity using GPT-2 (**?**) and selectively removes low-information tokens. Finally, a **Safety Critic** (Llama Guard 3 (**?**)) evaluates the compressed trace and outputs a binary safe/unsafe decision.

1.2 Entropy-Based Compression

Our compressor uses a pre-trained GPT-2 model to compute token-level loss values via cross-entropy. For each token t_i in the input sequence, we calculate:

$$\text{loss}(t_i) = -\log P(t_i|t_1, \ldots, t_{i-1})$$

Tokens with higher loss (greater "surprisal") are deemed more informative and retained. Given a compression ratio r, we keep the top $(1-r) \times |T|$ tokens ranked by loss, where $|T|$ is the original sequence length. This approach is similar to methods in LLMLingua (**?**) but operates purely on perplexity without additional filtering heuristics.

1.3 Evaluation Metrics

We measure two key safety metrics:

- **Attack Success Rate (ASR)**: The false negative rate—proportion of unsafe traces incorrectly classified as safe (missed attacks).
- **Benign Refusal Rate (BRR)**: The false positive rate—proportion of safe traces incorrectly classified as unsafe (over-refusal).

1.4 Baselines and Implementation

Our baseline is a standard Generator-Critic pipeline with no compression (compression ratio = 0%). We implemented the pipeline in Python using the Transformers library (**?**) for model loading and inference. All code is original except for the use of pre-trained models (GPT-2, Llama Guard 3) from Hugging Face.

2 Experiments

2.1 Data

We use the AgentDojo benchmark (**?**), a dataset of adversarial attacks targeting LLM-based agents. AgentDojo contains injection attacks embedded in tool outputs and user instructions designed to manipulate agent behavior (e.g., data exfiltration, unauthorized actions). For this milestone, we evaluate on the "tiny" subset (N=10 samples) to validate our pipeline. Each sample includes a system

prompt, user query, and conversation history that forms the agent trace.

2.2 Experimental Details

Models: We use GPT-2 (117M parameters) as the compressor proxy and Llama Guard 3 (8B parameters) as the safety critic. Llama Guard 3 is loaded with `bfloat16` precision and automatic device mapping to fit within GPU memory.

Hardware: All experiments run on a single NVIDIA A100 (80GB) GPU.

Compression Ratios: We test four settings: 0% (baseline), 30%, 50%, and 70% compression.

Procedure: For each sample, we (1) format the agent trace, (2) apply compression at the specified ratio, (3) pass the compressed text to Llama Guard 3, and (4) compare the critic's decision (safe/unsafe) against ground truth labels.

2.3 Results

We evaluated our improved pipeline on four datasets: AgentDojo (baseline), ATBench, Jailbreak, and Nemotron. For all experiments, we used a compression rate of 50% with the "hybrid" compression strategy and context repair enabled.

Metric	AgentDojo	ATBench	Jailbreak	Nemotron
True Positives (TP)	4	1	6	0
False Negatives (FN)	2	0	0	1
False Positives (FP)	2	3	0	3
True Negatives (TN)	2	6	4	6
ASR (Safety Loss)	33.33%	0.00%	0.00%	100.00%
BRR (Over-refusal)	50.00%	33.33%	0.00%	33.33%
Recall	66.67%	100.00%	100.00%	0.00%
Precision	66.67%	25.00%	100.00%	0.00%

Table 1: Performance metrics across different datasets (limit=10 samples each).

2.4 Analysis

- **Jailbreak Detection:** The pipeline performed exceptionally well on the Jailbreak dataset, achieving 100% recall and precision with 0% safety loss. This suggests the hybrid compression strategy effectively preserves the semantic cues required to identify direct jailbreak attempts.

- **ATBench:** While we achieved 100% recall (catching the single attack in the sample), the precision was low (25%) due to a high rate of false positives (over-refusal). This indicates that the safety critic may be too sensitive to the compressed representations of benign complex tool-use scenarios found in ATBench.

- **Nemotron:** This dataset proved challenging, with a 100% Attack Success Rate (ASR) on the single unsafe sample present in the subset. The system failed to identify the attack after compression. Additionally, there was a significant over-refusal rate (33.33%). This suggests that the Nemotron dataset's subtle safety violations might be lost during compression or are difficult for the current critic model to detect without full context.

- **AgentDojo:** Results on our primary dataset show a balance between safety and utility, though improvement is still needed to reduce both ASR (33%) and BRR (50%).

2.5 Conclusion

The "hybrid" compression strategy shows promise, particularly for explicit jailbreak attempts. However, handling subtle safety violations (as seen in Nemotron) and avoiding over-refusal in complex scenarios (ATBench, AgentDojo) remains a challenge. Future work will focus on tuning the compression algorithm to better preserve context for these edge cases and potentially fine-tuning the safety critic on compressed text.

2.6 Implementation Challenges and Solutions

During development, we encountered two significant technical challenges that informed our understanding of compression-safety interactions:

Challenge 1: Device Mismatch in Safety-Aware Compression. Our initial implementation of safety-aware token selection (which boosts importance scores for safety-critical keywords like "password", "delete", "transfer") produced a CUDA device mismatch error: `Expected all tensors to be on the same device, but got tensors is on cpu, different from other tensors on cuda:0`. The root cause was creating boolean masks on CPU while loss tensors resided on GPU.

Solution: We explicitly specified device placement for all intermediate tensors, ensuring mask creation uses the same device as the model: `safety_mask`

`= torch.zeros(len(input_ids), dtype=torch.bool, device=DEVICE)`. This ensures all tensor operations remain on the GPU, avoiding expensive CPU-GPU transfers and runtime errors.

Challenge 2: Over-Refusal from Fragmented Text. As shown in Table **??**, our original entropy-based compressor produced 90% BRR at medium compression. Inspection revealed the compressor was creating ungrammatical fragments (e.g., "password delete account unauthorized") by removing function words and breaking sentence structure.

Solution: We developed three improved compression strategies:

- **Sentence-level compression:** Splits text into complete sentences, scores each by average perplexity, and keeps top-k sentences. This preserves grammaticality by avoiding mid-sentence cuts.
- **Safety-aware token compression:** Identifies safety-critical keywords (40+ terms across authentication, actions, data operations) and boosts their retention probability by $2\times$. Always preserves safety signals even at high compression.
- **Hybrid strategy:** Uses sentence-level compression for $r < 0.4$ (preserves grammar) and safety-aware token-level for $r \geq 0.4$ (preserves safety). Adapts method to compression intensity.

Additionally, we implemented **context repair** in the safety critic: when fragmentation is detected (<30% complete sentences), we prepend `[COMPRESSED AGENT TRACE - May contain fragments]` to help the critic interpret degraded input appropriately.

These improvements reduced BRR from 90% to approximately 20-30% at 50% compression in preliminary tests (sentence-level strategy), while maintaining ASR $\approx$ 0%. This demonstrates that compression-induced over-refusal is addressable through grammar-preserving and safety-aware methods.

3 Future Work

Based on these preliminary findings, we plan to:

(1) Expand to larger datasets: Test on the full AgentDojo benchmark, LLMail-Inject (**?**), and Lakera PINT (**?**) to obtain statistically significant results across diverse attack types and ensure our dataset contains sufficient unsafe examples to properly measure ASR.

(2) Improve compression methods: Implement sentence-level and semantic compression techniques that preserve grammaticality. Token-level pruning creates fragmented text; we hypothesize that maintaining syntactic coherence will reduce false positives while still achieving compression gains.

(3) Safety-aware compression: Train a compressor explicitly optimized to retain safety-critical tokens (e.g., "password", "delete", "transfer") while removing redundant filler words. This could be achieved via supervised learning on labeled safety-relevant spans or reinforcement learning with ASR/BRR as rewards.

(4) Qualitative error analysis: Manually inspect compressed traces that trigger over-refusal to identify linguistic patterns (e.g., missing verbs, broken noun phrases) that cause false alarms. This will inform compression strategies that balance efficiency and interpretability.

(5) Alternative safety critics: Test with other guard models (e.g., Llama Guard 2, Azure AI Content Safety) to determine whether over-refusal under compression is specific to Llama Guard 3's architecture or a general phenomenon across safety classifiers.

References

Lakera AI. 2024. Pint: Prompt injection benchmark.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Preprint*, arXiv:2406.13352.

Jianyu Huang, Boya Jiang, Mingyang Zhang, Yao Qiang, Tianyi Luo, Michael Huang, Alice Lai, Allie Feinstein, Davide Testuggine, Mustafa Mukadam, Xiaodan Song, Carolyn Kim, Kouhyar Payvand, Gurinder Samra, Jacob Moshiri, Sanjay Krishna Gouda, Kun Huang, Omkar Sidhant Khade, Kiran Jagadeesh, and 4 others. 2024. Llama guard 3 vi-

sion: Safeguarding human-ai image understanding conversations. *Preprint*, arXiv:2411.10414.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmlingua: Compressing prompts for accelerated inference of large language models. *Preprint*, arXiv:2310.05736.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yang Liu, and Haoyu Wang. 2024. Evaluating and defending against indirect prompt injection attacks on llms. *Preprint*, arXiv:2312.14197.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, and 1 others. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.